

Documentation technique

Data Lake financier

Groupe : Artemiy Smogunov, Nathan Smadja-Tubiana, Patrice Ignongui

Projet EFREI 2025-2026

Guide d'installation et d'utilisation

Ce guide part d'un clone neuf du dépôt. À la fin, la stack doit être active, le DAG doit avoir terminé et les trois zones doivent être consultables avec FastAPI.

1. Vérifier les prérequis

```
docker --version
docker compose version
uv --version
```

Le projet utilise Python 3.11. uv lit cette version dans `.python-version`. S'il ne trouve pas Python 3.11 sur la machine, il peut télécharger une version gérée dans son propre cache.

2. Créer l'environnement Python

Depuis la racine du dépôt :

```
uv sync --frozen --all-groups
```

`--frozen` interdit à uv de modifier le verrou pendant l'installation. `--all-groups` installe les groupes API, tests et documentation en plus des dépendances du pipeline.

Contrôles :

```
uv lock --check
uv run python --version
uv run pytest -q
```

La validation finale retourne 14 `passed`. Un avertissement pandas annonce que PyArrow deviendra obligatoire dans pandas 3.0. Le projet utilise pandas 2.2.0 et n'appelle pas PyArrow, cet avertissement n'empêche donc aucun test actuel.

3. Construire et démarrer la stack

```
docker compose up -d --build
```

Le build FastAPI crée un environnement dans `/opt/venv` à partir de `uv.lock`. Le build Airflow part de `apache/airflow:slim-2.8.1-python3.11`. Il installe les dépendances métier avant le démarrage des conteneurs. Les services Airflow ne téléchargent donc plus pandas, yfinance ou scikit-learn à chaque redémarrage.

Suivi du démarrage :

```
docker compose ps -a
docker compose logs --tail 100 airflow-init api airflow-scheduler
```

`minio-init` crée les buckets `raw-financial-data` et `raw-api-data`. `airflow-init` migre la base Airflow puis crée l'utilisateur local. Ces deux services se terminent ensuite normalement.

4. Contrôler les stockages

```
curl http://localhost:8000/health
```

Réponse attendue :

```
{
  "overall": "ok",
  "services": {
    "postgresql": {"status": "ok", "details": null},
    "minio": {"status": "ok", "details": "Buckets : ['raw-api-data', 'raw-financial-data']"},
    "elasticsearch": {"status": "ok", "details": "version 8.11.0"}
  }
}
```

Financial Data Lake API 1.0.0 QAS 3.1

/openapi.json

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit deux endpoints d'ingestion.

The screenshot shows the Swagger UI interface for the 'Health' endpoint. The endpoint is a GET request to '/health' for 'Vérification de l'état des services'. The description states: 'Vérifie la connectivité de tous les services : PostgreSQL, MinIO et Elasticsearch.' The 'Parameters' section is empty. The 'Responses' section shows a 200 status code with a JSON response body. The JSON body contains the status of various services: PostgreSQL (ok), MinIO (ok, with buckets: ['raw-api-data', 'raw-financial-data']), and Elasticsearch (ok, version 8.11.0). The 'Request URL' is 'http://localhost:18000/health'. The 'Server response' section shows the status code 200 and the response body. The 'Response headers' section shows 'content-length: 217' and 'content-type: application/json'.

Contrôle de santé dans Swagger

La capture confirme le code HTTP 200 et les trois connexions. Elle ne prouve pas que le pipeline a déjà chargé des données. Pour cela, il faut consulter /stats après un run.

5. Déclencher le DAG

Airflow est disponible sur <http://localhost:8080> avec admin / admin.

Le DAG est en pause lors de sa première création. Les commandes suivantes l'activent et lancent un run manuel :

```
docker compose exec airflow-webserver airflow dags unpause financial_data_lake_pipeline
docker compose exec airflow-webserver airflow dags trigger financial_data_lake_pipeline
```

Suivi en ligne de commande :

```
docker compose exec airflow-webserver airflow dags list-runs -d
financial_data_lake_pipeline
```

Airflow DAGs Cluster Activity Datasets Security Browse Admin Docs 08:04 UTC AU

DAGs

All 1 Active 1 Paused 0 Running 0 Failed 0 Filter DAGs by tag Search DAGs Auto-refresh

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions
 financial_data_lake_pipeline data-lake finance yfinance	data-engineer		06***15	2026-07-12, 07:59:43	2026-07-13, 06:00:00		 

Showing 1-1 of 1 DAGs

Version: v2.8.1
Git Version: .release:c0ffa9c5d96625c68ded9562632674ed366b5eb3

DAG actif dans Airflow

Le DAG est actif et planifié à 6 h UTC du lundi au vendredi. La colonne du dernier run affiche l'exécution manuelle utilisée pour cette documentation.

financial_data_lake_pipeline Pipeline complet Raw→Staging→Curated pour le data lake financier

Schedule: 0 6 * * 1-5 | Next Run: 2026-07-13, 06:00:00

Grid | Graph | Calendar | Task Duration | Task Times | Landing Times | Gantt | Details | Code | Audit Log

12/07/2026 08:06:42 | 25 | All Run Types | All Run States | Clear Filters | Auto-refresh

Press **shift** + **/** for Shortcuts

deferred failed queued removed restarting running scheduled skipped success up_for_reschedule up_for_rely upstream_failed no_status

financial_data_lake_pipeline

Details | Graph | Gantt | Code

DAG Runs Summary

Total Runs Displayed	1
Total success	1
First Run Start	2026-07-12, 07:59:42 UTC
Last Run Start	2026-07-12, 07:59:42 UTC
Max Run Duration	00:00:49
Mean Run Duration	00:00:49
Min Run Duration	00:00:49

DAG Summary

Total Tasks	7
EmptyOperators	2
PythonOperators	5

Tâches du run dans la vue Grid

La vue Grid permet de contrôler les sept tâches. Une grille verte indique que les opérateurs se sont terminés, mais il faut aussi lire `log_summary` pour connaître les erreurs partielles collectées par les fonctions d'ingestion.

Pour lire le résumé, ouvrir le run dans Airflow, sélectionner la tâche

`log_summary`, puis ouvrir son journal. Cette tâche affiche le nombre de succès et d'erreurs pour les deux ingestions, ainsi que le nombre de lignes présentes dans les zones Staging et Curated.

6. Utiliser Swagger

Swagger est disponible sur <http://localhost:8000/docs>.

Financial Data Lake API 1.0.0 OAS 3.1

/openapi.json

API Gateway du data lake financier (Yahoo Finance / yfinance). Expose les trois zones du data lake (Raw, Staging, Curated) et fournit deux endpoints d'ingestion.

Health		^
GET	/health	Vérification de l'état des services
Stats		^
GET	/stats	Métriques de remplissage du data lake
Raw		^
GET	/raw	Données brutes de la zone Raw
GET	/raw/objects	Liste des objets MinIO dans la zone Raw
Staging		^
GET	/staging	Données staging avec indicateurs techniques
GET	/staging/tickers	Liste des tickers disponibles en staging
Curated		^
GET	/curated	Données curated avec scores d'anomalie
GET	/curated/anomalies/summary	Résumé des anomalies par ticker
GET	/curated/signals	Tickers avec signaux actifs (buy/sell)
Ingest		^
POST	/ingest	Ingestion synchrone avec benchmark
Ingest Fast		^
POST	/ingest_fast	Ingestion parallèle

Ensemble des routes FastAPI

Les routes sont séparées par zone. Une route peut être ouverte, puis exécutée avec le bouton Try it out. Swagger affiche la commande curl, l'URL, le code HTTP et le JSON renvoyé.

7. Lire la zone Raw

```
curl "http://localhost:8000/raw?ticker=AAPL&limit=1"
```

Filtres acceptés :

- ticker pour un symbole précis ;
- source=file, source=api ou source>manual ;
- from_date et to_date au format YYYY-MM-DD ;
- limit entre 1 et 1000.

Liste des objets MinIO :

```
curl "http://localhost:8000/raw/objects?bucket=api&ticker=AAPL"
```

```
{
  "total": 733,
  "returned": 1,
  "documents": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "open": 314.720001220703,
      "high": 316.910003662109,
      "low": 312.170013427734,
      "close": 315.320007324219,
      "volume": 34109200,
      "source": "yfinance_api",
      "metadata": {
        "shortName": "Apple Inc.",
        "sector": "Technology",
        "industry": "Consumer Electronics",
        "marketCap": 4631217307648,
        "currency": "USD",
        "exchange": "NYS"
      },
      "ingested_at": "2026-07-12T07:59:55.428306+00:00"
    }
  ]
}
```

Objet AAPL lu dans Raw

La réponse montre le cours reçu, la source `yfinance_api`, les métadonnées Yahoo Finance et l'heure d'ingestion. Raw ne contient pas encore les indicateurs techniques.

8. Lire la zone Staging

```
curl "http://localhost:8000/staging?ticker=AAPL&limit=1"
curl http://localhost:8000/staging/tickers
```

```
{
  "total": 733,
  "returned": 1,
  "offset": 0,
  "limit": 1,
  "data": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "open": "314.720001",
      "high": "316.910004",
      "low": "312.170013",
      "close": "315.320007",
      "adj_close": null,
      "volume": 34109200,
      "sma_20": "232.351783",
      "sma_50": "230.455832",
      "ema_12": "244.307807",
      "ema_26": "235.898777",
      "rsi_14": "92.2640",
      "macd": "8.409029",
      "macd_signal": "2.210671",
      "bollinger_upper": "272.366227",
      "bollinger_lower": "192.397338",
      "daily_return": "0.328614",
      "volatility_20": "0.073249",
      "ingested_at": "2026-07-12 08:00:16.427784"
    }
  ]
}
```

Ligne AAPL dans Staging

La ligne contient les colonnes financières normalisées et les indicateurs calculés. La clé unique est (ticker, date). Un nouveau run met à jour une date existante au lieu d'ajouter un doublon.

9. Lire la zone Curated

```
curl "http://localhost:8000/curated?ticker=AAPL&limit=1"
curl http://localhost:8000/curated/anomalies/summary
curl http://localhost:8000/curated/signals
```



```
{
  "total": 733,
  "returned": 1,
  "offset": 0,
  "limit": 1,
  "data": [
    {
      "ticker": "AAPL",
      "date": "2026-07-10",
      "close": "315.320007",
      "volume": 34109200,
      "daily_return": "0.328614",
      "volatility_20": "0.073243",
      "rsi_14": "92.2640",
      "anomaly_score": "-0.740538",
      "is_anomaly": true,
      "anomaly_type": "price_spike",
      "price_trend": "bullish",
      "signal": "hold",
      "processed_at": "2026-07-12 00:00:26.838875"
    }
  ]
}
```

Ligne AAPL dans Curated

Cette ligne est marquée comme anomalie price_spike. Le champ price_trend vaut bullish et le signal vaut hold. Le signal reste neutre parce que les conditions RSI et MACD nécessaires à buy ou sell ne sont pas réunies ensemble.

10. Comprendre les compteurs

```
curl http://localhost:8000/stats
```

```
{
  "raw_minio": {
    "bucket_file_objects": 1,
    "bucket_api_objects": 14,
    "total_objects": 15
  },
  "raw_elasticsearch": {
    "total_documents": 746,
    "by_ticker": {
      "AAPL": 733,
      "AMZN": 1,
      "BRK-B": 1,
      "GOOGL": 1,
      "JNJ": 1,
      "JPM": 1,
      "META": 1,
      "MSFT": 1,
      "NVDA": 1,
      "TSLA": 1,
      "DJIA": 1,
      "GSPC": 1,
      "IXIC": 1,
      "RUT": 1
    }
  },
  "staging": {
    "total_rows": 746,
    "by_ticker": [
      {
        "ticker": "AAPL",
        "rows": 733,
        "from": "2022-01-03",
        "to": "2026-07-10"
      },
      {
        "ticker": "AMZN",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "BRK-B",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "DJIA",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "GOOGL",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "GSPC",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "IXIC",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "JNJ",
        "rows": 1,
        "from": "2026-07-10",
        "to": "2026-07-10"
      },
      {
        "ticker": "JPM"
      }
    ]
  }
}
```

Raw et Staging dans la réponse stats

```

Impression élégante ☒
{
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "JPM",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "META",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "MSFT",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "NVDA",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "^RUT",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
},
{
  "ticker": "TSLA",
  "rows": 1,
  "from": "2026-07-10",
  "to": "2026-07-10"
}
],
"curated": {
  "total_rows": 746,
  "anomalies_detected": 37,
  "anomaly_rate_pct": 4.96,
  "by_ticker": [
    {
      "ticker": "AAPL",
      "rows": 733,
      "anomalies": 37,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "AMZN",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "BRK-B",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "^DJI",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "GOOGL",
      "rows": 1,
      "anomalies": 0,
      "last_date": "2026-07-10"
    },
    {
      "ticker": "^GSPC",

```

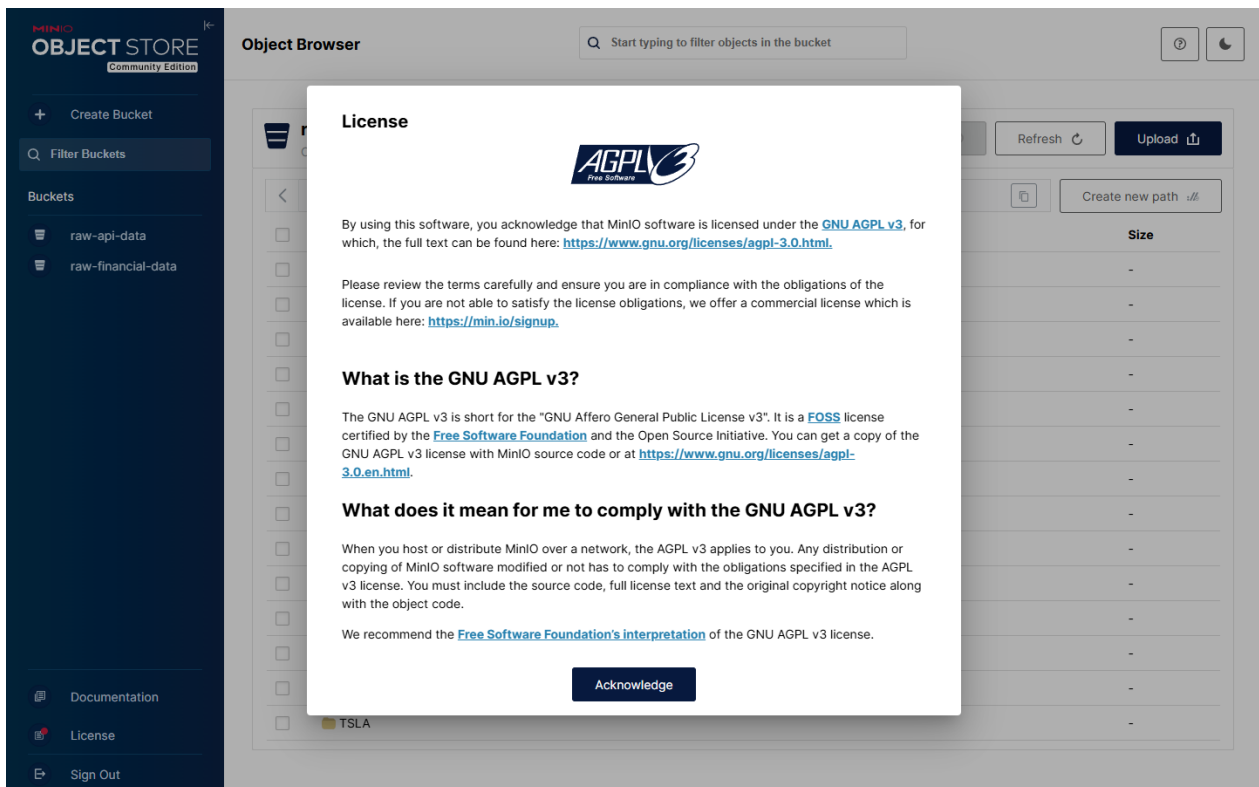
Curated dans la réponse stats

Sur les volumes vides de la validation finale, un run produit 15 objets MinIO. Le bucket fichier reçoit un CSV AAPL. Le bucket API reçoit quatorze JSON, un par ticker. Elasticsearch contient 746 documents. Staging et Curated contiennent chacun 746 lignes.

La table `ingestion_logs` est vide après un run Airflow. Le code écrit dans cette table lors des appels manuels à `/ingest` et `/ingest_fast`, pas lors des tâches du DAG.

11. Examiner MinIO

La console est disponible sur `http://localhost:9001` avec `minioadmin / minioadmin`.



Buckets et dossiers API dans MinIO

Le panneau gauche montre les deux buckets. Le bucket API contient un dossier pour chacun des quatorze tickers. Les noms d'objets incluent le ticker, la date du run et l'heure d'ingestion.

12. Lancer une ingestion manuelle

Mode séquentiel :

```
curl -X POST http://localhost:8000/ingest \
  -H "Content-Type: application/json" \
  -d '{"data":{"tickers":["AAPL","MSFT"],"period":"3mo","run_staging":true,"run_curated":true}}'
```

Mode parallèle :

```
curl -X POST http://localhost:8000/ingest_fast \
  -H "Content-Type: application/json" \
  -d '{"data":{"tickers":["AAPL","MSFT"],"period":"3mo","run_staging":true,"run_curated":true}}'
```

La réponse donne le temps de chaque étape et les erreurs par ticker. Le statut peut valoir `partial` tout en conservant un code HTTP 200. Il faut donc lire le tableau `errors` et pas seulement le code HTTP.

13. Arrêter ou réinitialiser

```
docker compose down
```

Cette commande conserve les volumes.

```
docker compose down -v
```

Cette commande supprime toutes les données locales du projet. Elle est utilisée avant une exécution de validation sur volumes vides.

14. Régénérer les PDF

```
uv run --group docs python scripts/generate_pdf_deliverables.py
```

Le script produit le rapport technique PDF et le guide d'utilisation PDF dans livrables.

15. Problèmes courants

Si un port est déjà occupé, `docker compose up` échoue avec un message de liaison de port. Il faut arrêter le service qui utilise ce port ou choisir un autre mapping pour l'environnement de test.

Si le DAG n'apparaît pas :

```
docker compose exec airflow-webserver airflow dags list-import-errors
```

Si `/health` retourne `degraded`, consulter les logs du service concerné :

```
docker compose logs --tail 200 postgres minio elasticsearch api
```

Si les compteurs restent à zéro, vérifier que le DAG a été activé, déclenché et terminé. Un démarrage de la stack ne lance pas automatiquement le DAG, car il est créé en pause.